



Работа с файлами



Основные функции работы с файлами

Описание	Сигнатура	Комментарий
Открыть файл	<code>FILE *fopen(const char *path, const char *mode);</code>	Буферизованный I/O (glibc FILE)
Закрыть файл	<code>int fclose(FILE *stream);</code>	Делает <code>fflush</code> + закрывает <code>fd</code>
Чтение из файла	<code>size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);</code>	Буферизованное чтение (прочитать <i>nmemb</i> элементов, каждый размером <i>size</i> , из <i>stream</i> → в <i>ptr</i>)
Запись в файл	<code>size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);</code>	Буферизованная запись
Позиция в файле	<code>long ftell(FILE *stream);</code>	Возвращает текущий <code>offset</code>
Установка позиции	<code>int fseek(FILE *stream, long offset, int whence);</code>	<code>SEEK_SET</code> / <code>SEEK_CUR</code> / <code>SEEK_END</code>
Сброс буфера	<code>int fflush(FILE *stream);</code>	Сбрасывает <code>userspace</code> buffer в <code>kernel</code>
Проверка EOF	<code>int feof(FILE *stream);</code>	Проверяет конец файла
Проверка ошибки	<code>int ferror(FILE *stream);</code>	Проверяет ошибки I/O
Получить <code>fd</code>	<code>int fileno(FILE *stream);</code>	Достаёт <code>underlying fd</code>



Чтение/запись

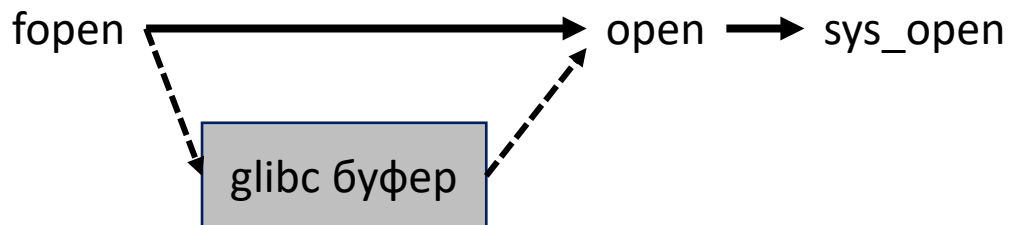
Уровень	Функция	Описание	Плюсы	Минусы
Linux syscall	<code>read(int fd, void *buf, size_t count)</code>	Чтение байтов напрямую из ядра	минимальный overhead, предсказуемость	нет буферизации, больше syscalls
	<code>write(int fd, const void *buf, size_t count)</code>	Запись напрямую в ядро	быстро, просто	нет форматирования, нет буфера
stdio	<code>int getc(FILE *stream)</code>	Чтение 1 символа из потока. Обычно - макрос	очень быстро при заполненном буфере	хуже для массовых данных
	<code>int putc(int c, FILE *stream)</code>	Запись 1 символа в поток	быстро через буфер FILE	неэффективно для больших объёмов
	<code>fread(void *ptr, size_t size, size_t nmemb, FILE *stream)</code>	Блочное буферизованное чтение	меньше syscalls, быстрые массовые операции	сложнее контроль, двойная буферизация
	<code>fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream)</code>	Блочная запись	буферизация, эффективно для больших данных	не сразу в файл (нужен fflush)
	<code>fgets(char *s, int size, FILE *stream)</code>	Чтение строки до \n или лимита	удобно для текста	медленнее, работает посимвольно логически
	<code>fputs(const char *s, FILE *stream)</code>	Запись строки	проще чем fwrite для текста	нет форматирования
	<code>fprintf(FILE *stream, const char *fmt, ...)</code>	Форматированная запись	удобно (printf-style)	медленнее (format parsing + buffer)
	<code>fscanf(FILE *stream, const char *fmt, ...)</code>	Форматированное чтение	удобно для простых форматов	медленно, хрупко

И это не все...



fopen

userspace



kernelspace

```
struct proc {  
    int pid; // PID  
    pagetable_t pagetable; // Таблица страниц пользователя  
    struct file *ofile[NOFILE]; // Открытые файлы  
    struct inode *cwd; // Текущая директория  
    char name[16]; // Имя процесса (отладка)  
    ...  
};
```



```
struct file {  
    ...  
    struct inode *ip; // Указатель на inode  
    unsigned int f_flags; // Флаги, указанные при вызове open  
    mode_t f_mode; // Режим доступа к файлу  
    loff_t f_pos; // Текущий указатель файла (смещение)  
    struct fown_struct f_owner; // Информация о владельце  
    ...  
};
```

```
struct inode {  
    ...  
    kuid_t i_uid; // ID владельца  
    struct super_block *i_sb; // Указатель на суперблок  
    loff_t i_size; // Размер файла  
    time64_t i_atime_sec; // Время доступа к файлу  
    ...  
}
```

Список в ядре
из открытых файлов



Работа с файлами

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE* fp;
```

```
    fp = fopen("data.txt", "w");
```

```
    fprintf(fp, "%s", "Hello!");
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```



data.txt – Блокнот

Файл Правка Формат Вид Справка

Hello!



Работа с файлами

```
#include <stdio.h>
```

```
int main()
{
    FILE* fp;
    char sym[10];
    fp = fopen("data.txt", "w");
    fprintf(fp, "%s", "Hello world!");
    fclose(fp);

    fp = fopen("data.txt", "r");
    while ((fgets(sym, 10, fp)) != NULL)
    {
        printf("%s\n", sym);
    }
    fclose(fp);
    return 0;
}
```



data.txt – Блокнот

Файл Правка Формат Вид Справка

Hello world!

Hello wor
ld!



Работа с файлами

```
fp = fopen("data1.txt", "r");  
if(fp == NULL){  
    perror("fopen");  
    return 1;  
}
```

При работе с файлами
рекомендуется проводить
проверку на корректное
открытие

Полный код см. в заметках к слайду



Буфер stdout

```
#include <stdio.h>

int main()
{
    fprintf(stdout, "Hello 1 ");
    fprintf(stderr, "This is error. ");
    fprintf(stdout, " Hello 2 \n");
    return 0;
}
```

Что будет выведено в результате работы программы?



Буфер stdout

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    fprintf(stdout, "Hello 1 ");
```

```
    fprintf(stderr, "This is error. ");
```

```
    fprintf(stdout, " Hello 2 \n");
```

```
    return 0;
```

```
}
```

Что будет выведено в результате работы программы?

```
sab@LAPTOP-B03PIUAN:.../Lesson4$ ./ffl
```

```
This is error. Hello 1 Hello 2
```



Буфер stdout

```
#include <stdio.h>
#include <assert.h>
#include <stdbool.h>
```

```
int main()
{
    printf("123");
    assert(false);
    return 0;
}
```

Что будет выведено в результате работы программы?

```
a.out: test.c:6: main: Assertion '0' failed.
Aborted (core dumped)
```



Перенаправление потоков ввода/вывода

```
#include <stdio.h>                                     write.c

int main(){
    printf("world!");
    return 0;
}
```

```
#include <stdio.h>                                     read.c

int main(){
    char text[100];
    printf("Hello ");
    scanf("%s", text);
    printf("%s\n", text);
    return 0;
}
```

```
./read > log.txt
./write > text.txt
./read < text.txt
./read > log.txt < text.txt
./write | ./read
```



Строки



Строки в Си

Строк в Си НЕТ



Строки в Си

Строки == Массив символов

```
#include <stdio.h>

const int N = 10;

int main()
{
    char name[N];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

```
Enter name: SAB
Your name is SAB.
```



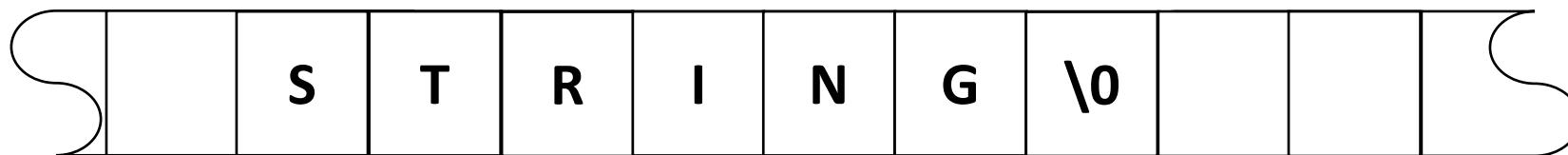
```
Enter string: SAB
String is SAB.
String length is 3

Symbol      Code
S            0x53
A            0x41
B            0x42
             0x0
■            0xfffffffffe
■            0xfffffffffe
■            0xfffffffffe
■            0xfffffffffe
■            0xfffffffffe
■            0xfffffffffe
```

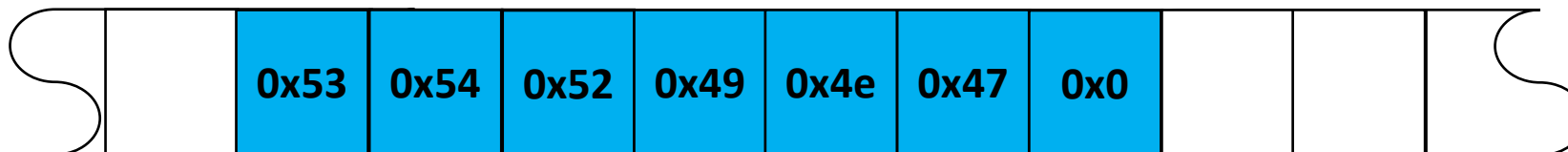


Как хранится строка в памяти

Символы



Коды символов





Чтение строк

Функция	Формат / Пример	Особенности
scanf	<code>scanf("%s", str);</code>	Читает до пробела, нужно ограничивать длину: <code>%99s</code>
fgets	<code>fgets(str, sizeof(str), stdin);</code>	Читает до конца строки или лимита. Включает <code>\n</code> .
gets	<code>gets(str);</code>	Опасно! Нет ограничения длины, удалена из C11
getchar	<code>while ((ch = getchar()) != '\n')</code>	Чтение символов по одному, нужно самостоятельно добавлять <code>\0</code>
getline (POSIX)	<code>getline(&line, &len, stdin);</code>	Динамическая память, возвращает количество символов, включая <code>\n</code> . Не стандарт C.



Запись строк

Функция	Формат / Пример	Особенности
<code>printf</code>	<code>printf("%s", str);</code>	Универсальная, можно выводить форматированные строки, числа, символы.
<code>puts</code>	<code>puts(str);</code>	Выводит строку + автоматически добавляет <code>\n</code> в конце.
<code>putchar</code>	<code>putchar('A');</code>	Выводит один символ за раз, удобно для циклов или поэтапного формирования вывода.
<code>fprintf</code>	<code>fprintf(stdout, "%s", str);</code>	Похож на <code>printf</code> , но позволяет выводить в файлы или потоки.
<code>fputs</code>	<code>fputs(str, stdout);</code>	Похож на <code>puts</code> , но не добавляет <code>\n</code> , можно выводить в файлы.



Полезные функции

```
#include <string.h>
```

Функция	Сигнатура	Описание
atoi	<code>int atoi(const char *nptr);</code>	Преобразует строку в <code>int</code> (без проверки ошибок, UB при некорректном вводе)
strcmp	<code>int strcmp(const char *s1, const char *s2);</code>	Лексикографически сравнивает строки
strcat	<code>char *strcat(char *dest, const char *src);</code>	Дописывает <code>src</code> в конец <code>dest</code> (ищет <code>\0</code> , затем копирует)
strstr	<code>char *strstr(const char *haystack, const char *needle);</code>	Ищет подстроку <code>needle</code> в <code>haystack</code> , возвращает указатель или <code>NULL</code>
strtok	<code>char *strtok(char *str, const char *delim);</code>	Разбивает строку на токены по разделителям



А еще много-много всего...

Функция	Назначение
memcpy	копирование памяти
memmove	копирование с учётом перекрытия
memset	заполнение байтами
memcmp	сравнение памяти
memchr	поиск байта
strlen	длина строки
strcpy	копирование строки
strncpy	копирование с ограничением
strcat	конкатенация
strncat	конкатенация с ограничением
strcmp	сравнение
strncmp	сравнение с ограничением
strchr	поиск символа
strrchr	поиск с конца
strstr	поиск подстроки
strspn	длина сегмента до токена НЕ из набора
strcspn	длина сегмента до токена из набора
strpbrk	поиск любого символа из набора
strtok	разбиение строки



Сравнение строк

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char str1[] = "hello";
    char str2[] = "world";

    int result = strcmp(str1, str2);

    if (result == 0)
        printf("Strings are equal\n");
    else if (result < 0)
        printf("str1 is less than str2\n");
    else
        printf("str1 is greater than str2\n");

    return 0;
}
```

str1 < str2 res = -1
str1 > str2 res = 1
str1 = str2 res = 0



Копирование строк

```
#include <stdio.h>
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[] = "hello";
```

```
    char str2[10] = {'\0'};
```

```
    char str3[10] = {'\0'};
```

```
    str[2]='\\0';
```

```
    memcpy(str2, str, 5);
```

```
    strcpy(str3, str);
```

```
    fwrite(str2, 10, 1, stdout);
```

```
    putc('\\n', stdout);
```

```
    fwrite(str3, 10, 1, stdout);
```

```
    putc('\\n', stdout);
```

```
    return 0;
```

```
}
```

Копирует до символа \\0

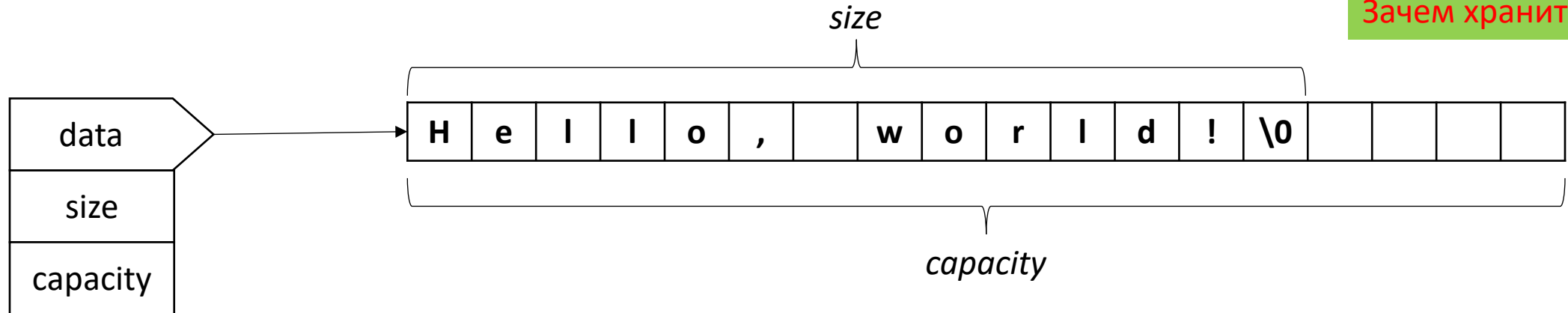
he
he

Просто printf нельзя – тоже до \\0
выводит по умолчанию

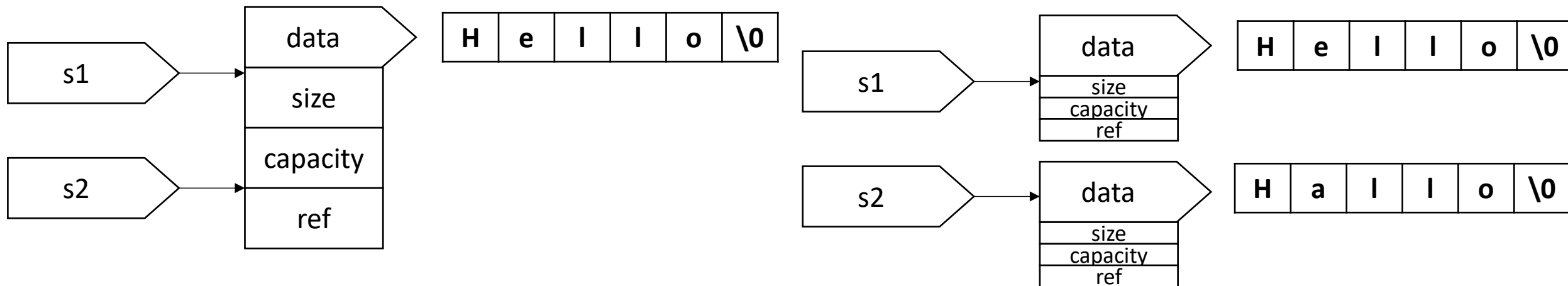


std::string

Зачем хранить \0?



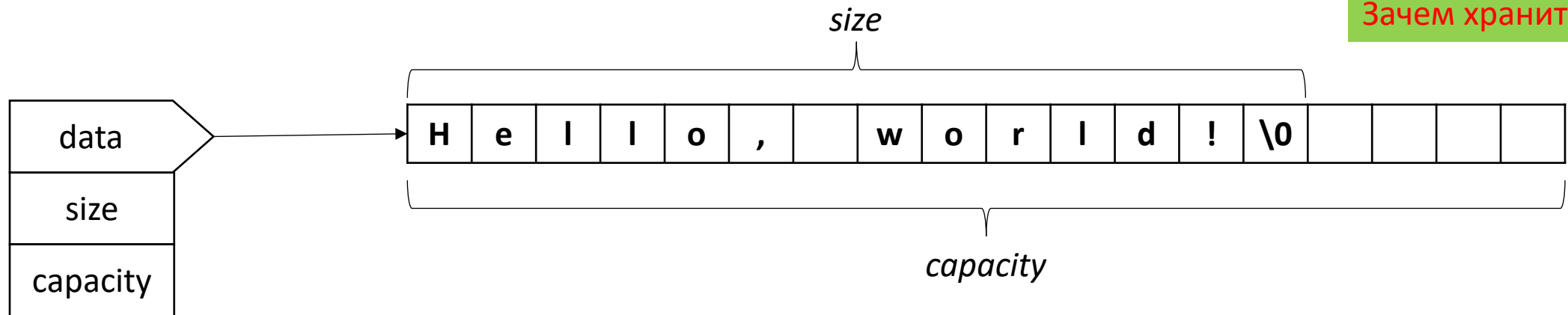
COW





std::string

Зачем хранить \0?



SSO

```
struct string {  
    size_t size;  
    size_t capacity;  
    union {  
        char *heap;    // длинные строки  
        char small[16]; // короткие строки  
    };  
};
```